

PART V

COMPLEXITY

Extended Edition

Formalizing the Big-O intuition into a rigorous, reflexive toolkit

Every earlier Part leaned on complexity intuition informally — "this is $O(n)$," "nested loops give $O(n^2)$." This Part makes that intuition rigorous and complete: formal asymptotic notation, recursion analysis via recursion trees and the Master Theorem, amortized analysis done properly, space complexity, and a long, deliberately extensive set of worked examples and practice drills so that reading a piece of code and stating its complexity becomes automatic, not effortful.

Contents of Part V

- 1. Why Complexity Analysis Matters (a deeper look)
- 2. Asymptotic Notation: Big-O, Big-Omega, Big-Theta
- 3. Analyzing Loops — the Complete Toolkit
- 4. Analyzing Recursion: Recursion Trees and the Master Theorem
- 5. Space Complexity and Auxiliary Space
- 6. Best Case, Average Case, and Worst Case
- 7. Amortized Analysis in Depth
- 8. Complexity of STL Containers and Algorithms — the Full Reference
- 9. Complexity Case Studies: Sorting, Searching, Graphs, Trees, DP
- 10. Hidden-Complexity Traps
- 11. Complexity Practice Drills (30 worked examples)
- Part V Revision, Master Complexity Cheat Sheet

Chapter 1 — Why Complexity Analysis Matters (a deeper look)

1.1 The operation budget, precisely

A modern judge machine executes on the order of 10^8 – 10^9 simple operations per second (additions, comparisons, array accesses). A typical time limit of 1–2 seconds therefore gives you an operation budget of roughly 10^8 to 2×10^9 . This budget is the entire reason complexity analysis matters practically, not just theoretically: it converts "is my algorithm fast enough" from a vague worry into an arithmetic check you can do in seconds, before writing a single line of code.

n	$O(n)$	$O(n \log n)$	$O(n^{1.5})$	$O(n^2)$	$O(n^3)$	$O(2^n)$
10	10	33	32	100	1,000	1,024
100	100	664	1,000	10,000	10^6	$\approx 1.3 \times 10^{30}$
1,000	1,000	9,966	31,623	10^6	10^9	astronomically large
100,000	10^5	$\approx 1.7 \times 10^6$	$\approx 3.2 \times 10^7$	10^{10}	10^{15}	—
1,000,000	10^6	$\approx 2 \times 10^7$	10^9	10^{12}	10^{18}	—

WARNING — Read this table before every problem with unfamiliar constraints

Notice the jump between $O(n \log n)$ and $O(n^2)$ around $n = 100,000$: roughly 1.7 million operations versus 10 billion. This single row is why "n up to $10^5 \rightarrow$ think $O(n \log n)$ " became the reflexive rule quoted throughout this book — the table above is where that rule actually comes from.

1.2 What complexity does NOT tell you

Big-O describes how running time scales as input size grows — it deliberately ignores constant factors and lower-order terms. Two $O(n)$ algorithms can have wildly different real running times: one might do 2 operations per element, another might do 200. This means complexity analysis rules out algorithms that are asymptotically too slow, but does not by itself guarantee that two algorithms with the same complexity class run in comparable real time. For borderline cases (an $O(n \log n)$ algorithm with a large constant, competing against a tight time limit), only an actual submission — or careful constant-factor reasoning — settles the question.

Chapter 2 — Asymptotic Notation: Big-O, Big-Omega, Big-Theta

2.1 Big-O — an upper bound

Informally: $f(n) = O(g(n))$ means f grows no faster than g , for large enough n , up to a constant factor. Formally, there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. In everyday use, "this algorithm is $O(n^2)$ " means its running time is at most proportional to n^2 , for large n .

Example: $f(n) = 3n^2 + 5n + 100$

We say $f(n) = O(n^2)$, because for large n , the $3n^2$ term dominates the $5n$ and the constant 100 .

Formally: pick $c = 8$ and $n_0 = 10$; for all $n \geq 10$, $3n^2 + 5n + 100 \leq 8n^2$.

2.2 Big-Omega — a lower bound, and Big-Theta — a tight bound

Notation	Meaning (informal)	Meaning (formal)
$O(g(n))$	f grows no faster than g	$f(n) \leq c \cdot g(n)$ for large n
$\Omega(g(n))$	f grows at least as fast as g	$f(n) \geq c \cdot g(n)$ for large n
$\Theta(g(n))$	f grows exactly as fast as g	$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for large n

NOTE — Why this book (and most competitive programmers) say "O" for everything

Formally, saying an algorithm "is $O(n^2)$ " only claims an upper bound — the true running time could even be smaller. In casual usage (including throughout this book, and in essentially all competitive programming discussion), " $O(n^2)$ " is used to mean $\Theta(n^2)$ — a tight description of the actual growth rate. This is a widely accepted looseness of language; the formal distinction matters mainly in academic algorithm analysis, and it's worth knowing it exists even though this book follows the common convention.

2.3 Rules for simplifying complexity expressions

- 1 Drop constant factors: $O(3n)$ is written $O(n)$.
- 2 Drop lower-order terms: $O(n^2 + n)$ is written $O(n^2)$, since n^2 dominates for large n .

- 3 Different variables for different input sizes must be kept separate: an algorithm doing work proportional to n for one array and m for another is $O(n + m)$, not $O(n)$ or $O(m)$ alone — do not conflate two independent input sizes into one variable.
- 4 Multiplication for nested/sequential dependent work, addition for sequential independent work — covered in full in Chapter 3.

Problem 2.1 — Simplify the complexity [Level 1]

An algorithm performs $4n^3 + 2n^2 \log n + 50n + 1000$ operations. What is its Big-O complexity?

Approach: $O(n^3)$ — the n^3 term dominates every other term for large n ; all lower-order terms and constants are dropped.

Chapter 3 — Analyzing Loops: the Complete Toolkit

3.1 Sequential (independent) loops: add

```
for (int i = 0; i < n; i++) { /* O(1) work */ } // O(n)
for (int i = 0; i < m; i++) { /* O(1) work */ } // O(m)
// Total:  $O(n) + O(m) = O(n + m)$ , NOT  $O(n)$  alone and NOT  $O(n*m)$ 
```

Two loops that run one after another (not nested) simply add their individual costs. This is easy to get wrong when n and m are visually similar-looking variables — always check whether two loops are sequential or nested before combining their costs.

3.2 Nested (dependent) loops: multiply

```
for (int i = 0; i < n; i++) // runs n times
    for (int j = 0; j < m; j++) // runs m times, for EACH i
        { /* O(1) work */ }
// Total: O(n * m)
```

This is the multiplicative counterpart of Section 3.1: because the inner loop's full cost is paid for every outer iteration, the costs multiply rather than add.

3.3 Triangular / dependent-bound nested loops

```
for (int i = 0; i < n; i++)
    for (int j = i; j < n; j++) // bound depends on i, not fixed at n
        { /* O(1) work */ }
```

The inner loop runs $n - i$ times for each i . Summing over all i from 0 to $n-1$ gives $n + (n-1) + (n-2) + \dots + 1 = n(n+1)/2$. Since constants and lower-order terms are dropped, this is $O(n^2)$ — the same complexity class as a full nested loop, just with roughly half the actual operations. This exact shape appears whenever you enumerate all pairs (i, j) with $i \leq j$, e.g. all subarrays (Part III).

Dry run: sum of inner-loop trip counts for $n = 5$

i	Inner loop range	Trip count
0	0..4	5
1	1..4	4
2	2..4	3
3	3..4	2
4	4..4	1

Total = $5+4+3+2+1 = 15 = n(n+1)/2$ for $n=5$. Still $O(n^2)$.

3.4 Loops with multiplicative or divisive steps: logarithmic complexity

```
int i = 1;
while (i < n) {
    i *= 2; // i doubles each iteration
}
// How many times does the loop run? log2(n) times – O(log n)
```

Whenever a loop variable is multiplied or divided by a constant factor each iteration (rather than incremented/decremented by a constant amount), the number of iterations is logarithmic in n . This is the same reasoning behind binary search's $O(\log n)$: each step eliminates a constant fraction of the remaining search space, not a constant count of elements.

Dry run: $n = 100$

Iteration	i before	i after
1	1	2
2	2	4
3	4	8
4	8	16
5	16	32
6	32	64
7	64	128 (loop ends, $128 \geq 100$)

7 iterations for $n = 100 \approx \log_2(100) \approx 6.6$, rounded up — matches the formula.

3.5 Nested loops where the inner loop is logarithmic

```
for (int i = 0; i < n; i++) { // n times
    int j = 1;
    while (j < n) j *= 2; // log n times, for each i
}
// Total:  $O(n \log n)$ 
```

NOTE — $O(n \log n)$ in practice

This exact shape — an outer $O(n)$ loop combined with an inner $O(\log n)$ operation — is precisely why binary-searching inside a loop over every element gives $O(n \log n)$ overall, one of the most common complexity classes in competitive programming (Part VI).

3.6 A loop whose body cost itself varies

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) sum += a[j]; //  $O(n)$  work, done n
    times
}
// Total:  $O(n)$  iterations *  $O(n)$  work each =  $O(n^2)$ 
```

WARNING — Watch for $O(n)$ work hidden inside what looks like $O(1)$ body

A single line like `if (find(v.begin(), v.end(), x) != v.end())` inside an $O(n)$ loop looks like one operation but is actually an $O(n)$ linear search — turning the whole loop into $O(n^2)$ silently. Chapter 10 catalogs this class of mistake in depth.

Chapter 4 — Analyzing Recursion: Recursion Trees and the Master Theorem

4.1 Recurrence relations

A recursive function's running time can be expressed as a recurrence relation: the cost of solving a problem of size n in terms of the cost of solving smaller subproblems, plus the extra work done outside the recursive calls.

```
long long factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
//  $T(n) = T(n-1) + O(1)$ ,  $T(1) = O(1)$ 
// Unrolled:  $T(n) = O(1) + O(1) + \dots$  ( $n$  times)  $= O(n)$ 
```

4.2 Recursion trees: visualizing where the work goes

A recursion tree draws every recursive call as a node, with children for each sub-call it makes. Summing the work done at every level of the tree gives the total complexity — this is often more intuitive than pure algebra, especially for recursion with more than one recursive call per step.

```
long long fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
//  $T(n) = T(n-1) + T(n-2) + O(1)$ 
```

Unlike factorial's single recursive call, naive Fibonacci branches into two calls per level. The recursion tree roughly doubles in width at each level and has depth n , giving $O(2^n)$ total nodes — an exponential blow-up caused entirely by recomputing the same subproblems (e.g. `fib(3)` is computed many times inside the tree for `fib(6)`). This exact inefficiency — overlapping subproblems recomputed from scratch — is the motivating example for memoization and dynamic programming (Part XV).

KEY IDEA — The recursion-tree habit

For any recursive function, ask: (1) how many recursive calls does one invocation make, and (2) how much does the input size shrink per call? Two calls with input roughly halved each time (like merge sort) behaves very differently from two calls with input reduced by only 1 each time (like naive Fibonacci) — the tree's shape, not just the branching factor alone, determines the total complexity.

4.3 The Master Theorem (for divide-and-conquer recurrences)

For recurrences of the form $T(n) = a \cdot T(n/b) + O(n^d)$ — meaning a subproblems, each of size n/b , plus $O(n^d)$ work to combine them — the Master Theorem gives the answer directly by comparing d against $\log_b(a)$:

Condition	Result
$d < \log_b(a)$	$T(n) = O(n^{(\log_b a)})$ — the recursive calls dominate
$d = \log_b(a)$	$T(n) = O(n^d \cdot \log n)$ — work is balanced across all levels
$d > \log_b(a)$	$T(n) = O(n^d)$ — the combine step dominates

Worked example: merge sort

Merge sort splits into $a = 2$ subproblems, each of size $n/2$ (so $b = 2$), and does $O(n)$ work to merge them (so $d = 1$). $\log_b(a) = \log_2(2) = 1 = d$, so we're in the "balanced" case: $T(n) = O(n^1 \cdot \log n) = O(n \log n)$ — matching the well-known result.

Worked example: binary search

Binary search makes $a = 1$ recursive call on a subproblem of size $n/2$ ($b = 2$), doing $O(1)$ work to decide which half to recurse into ($d = 0$). $\log_b(a) = \log_2(1) = 0 = d$, again balanced: $T(n) = O(n^0 \cdot \log n) = O(\log n)$.

$O(\log n)$.

WARNING — The Master Theorem only applies to this exact recurrence shape

It requires a single recursive call size (all a subproblems the same size n/b) and a polynomial combine cost. Recurrences with unequal subproblem sizes (like naive Fibonacci's $T(n-1) + T(n-2)$) need a recursion tree or another technique instead.

4.4 Space complexity of recursion: the call stack

Every recursive call pushes a new stack frame (holding its local variables and return address) onto the call stack, and pops it on return. This means a recursion's space complexity is proportional to its maximum depth, not its total number of calls. Naive Fibonacci makes $O(2^n)$ total calls but its stack depth is only $O(n)$ (the deepest chain of calls is $\text{fib}(n) \rightarrow \text{fib}(n-1) \rightarrow \dots \rightarrow \text{fib}(1)$), so its space complexity is $O(n)$ despite its exponential time complexity.

WARNING — Stack overflow from deep recursion

Each stack frame consumes real memory, and the call stack has a limited size (often a few MB, allowing on the order of 10^4 – 10^5 simple frames before crashing). A recursive function with $O(n)$ depth and n up to 10^6 can crash with a stack overflow even though its time complexity looks fine — this is a distinct failure mode from TLE or MLE, and a common trap when translating a naturally-recursive idea (like DFS) onto very large inputs without converting it to an iterative version.

Chapter 5 — Space Complexity and Auxiliary Space

5.1 Total space vs. auxiliary space

Total space includes the input itself; auxiliary space counts only the extra memory an algorithm uses beyond its input. Competitive programming almost always cares about auxiliary space, since the input's size is fixed regardless of which algorithm you choose.

```
long long sumArray(vector<int> &a) {  
    long long sum = 0; // O(1) auxiliary space – one variable,  
    regardless of n  
    for (int x : a) sum += x;  
    return sum;  
}  
  
vector<int> doubledCopy(vector<int> &a) {  
    vector<int> result; // O(n) auxiliary space – a new array  
    proportional to input size  
    for (int x : a) result.push_back(x * 2);  
    return result;  
}
```

5.2 In-place algorithms

An algorithm is in-place if its auxiliary space is $O(1)$ (sometimes $O(\log n)$ is also accepted as "effectively in-place," e.g. for recursion stack depth). The array-reversal, rotation, and duplicate-removal techniques from Part III are all in-place: they modify the input array directly rather than allocating a new one.

5.3 Time-space tradeoffs

Many optimizations trade auxiliary space for time. The Two Sum hash-set solution from Part II uses $O(n)$ auxiliary space to bring time down from $O(n^2)$ to $O(n)$; a prefix-sum array (Part III/Part VI) uses $O(n)$ space to answer range-sum queries in $O(1)$ instead of $O(n)$ each. Recognizing this tradeoff explicitly — "can I spend some memory to save time?" — is itself a reusable step in the improvement cycle from

Part II.

Technique	Time saved	Space spent
Hash set/map for existence checks	$O(n)$ instead of $O(n^2)$ for pair problems	$O(n)$
Prefix sums	$O(1)$ instead of $O(n)$ per range-sum query	$O(n)$
Memoization (Part XV)	Avoids recomputing overlapping subproblems	$O(\text{number of distinct states})$
Sparse table (Part XVIII)	$O(1)$ instead of $O(\log n)$ per idempotent range query	$O(n \log n)$

Chapter 6 — Best Case, Average Case, and Worst Case

6.1 Why the same algorithm can have different complexities

An algorithm's running time can depend on more than just input size — it can depend on the input's specific arrangement. Best case is the most favorable input, worst case is the least favorable, and average case is the expected running time over a typical/random distribution of inputs.

Example: linear search

Case	Scenario	Complexity
Best	Target is the first element	$O(1)$
Worst	Target is the last element, or absent	$O(n)$
Average	Target is equally likely at any position	$O(n)$ (expected $n/2$ comparisons, still $O(n)$)

6.2 Example: quicksort

Quicksort's behaviour depends heavily on pivot choice relative to the data. This is the single most commonly cited example in complexity theory for the best/average/worst distinction, and it directly explains a real competitive-programming gotcha (Chapter 9).

Case	Pivot behaviour	Complexity
Best / Average	Pivot roughly splits the array in half each time	$O(n \log n)$
Worst	Pivot is always the smallest or largest element (e.g. already-sorted input with a naive first-element pivot)	$O(n^2)$

6.3 Which case should you design for?

In competitive programming, always assume the worst case unless a problem's constraints or structure explicitly rule it out — judges' hidden test data is specifically constructed to trigger worst-case behaviour in common but fragile approaches (exactly like the anti-hash-hacking tests mentioned for `unordered_map` in Part IV). "It'll probably be fine on average" is not a safe design principle against an adversarial test setter.

Chapter 7 — Amortized Analysis in Depth

7.1 What amortized means

Amortized complexity describes the average cost per operation across a whole sequence of operations, even when individual operations vary wildly in cost. It is not the same as average-case analysis (Chapter 6): amortized analysis is a guarantee that holds for every sequence of

operations (not just typical/random ones) — the guarantee comes from the structure of the algorithm itself, not from assumptions about the input distribution.

7.2 Worked example: vector push_back (the accounting method)

When a vector's capacity is full, pushing one more element triggers a reallocation: a new block double the old capacity is allocated, and every existing element is copied over — an $O(\text{current size})$ operation. This seems to suggest `push_back` could be $O(n)$ in the worst case. Amortized analysis shows the average cost per push, across any sequence of n pushes starting from empty, is still $O(1)$:

Suppose capacity doubles: 1, 2, 4, 8, 16, ...
Reallocation happens when pushing the 2nd, 3rd (triggers grow to 4), 5th (grow to 8), 9th (grow to 16) ...
Total copying work for n pushes = $1 + 2 + 4 + 8 + \dots + n/2 < 2n$ (a geometric series)
Total work for n pushes (copying + the n pushes themselves) = $O(n) + O(n) = O(n)$
Average cost per push = $O(n) / n = O(1)$ <-- amortized $O(1)$

KEY IDEA — The accounting-method intuition

Think of every `push_back` as depositing a small fixed "credit" into an account, and every element pays for its own eventual copy during a future reallocation out of credit already saved up from earlier cheap pushes. Because capacity always at least doubles, the geometric series of copying costs never exceeds a constant multiple of n — this is exactly why the growth factor matters (growing by a fixed amount each time, instead of doubling, would make `push_back` genuinely $O(n)$ amortized, not $O(1)$).

7.3 Other amortized- $O(1)$ operations you already know

Operation	Why it's amortized rather than always $O(1)$
<code>vector::push_back</code>	Occasional $O(\text{current size})$ reallocation, averaged over a doubling schedule
Two-pointer / sliding-window total movement (Part III)	Each pointer moves forward at most n times total across the whole run, even though a single step of the outer loop can trigger many inner steps
Monotonic stack pushes/pops (Part IV)	Each index is pushed once and popped at most once across the entire run
<code>next_permutation</code> over a full cycle	Individual calls vary, but the total work across all $n!$ calls is $O(n! * n)$, giving $O(n)$ amortized per call

NOTE — The pattern behind all of these

In every case above, the proof works by bounding total work across the whole run, then dividing by the number of operations — rather than trying (and failing) to bound each individual operation's cost tightly. This is the single reusable idea of amortized analysis.

Chapter 8 — Complexity of STL Containers and Algorithms: the Full Reference

8.1 Containers

Container	Access	Insert/Erase (end)	Insert/Erase (middle)	Search
vector	$O(1)$	$O(1)$ amortized	$O(n)$	$O(n)$, or $O(\log n)$ if sorted
deque	$O(1)$	$O(1)$ both ends	$O(n)$	$O(n)$, or $O(\log n)$ if sorted
list	$O(n)$	$O(1)$	$O(1)$ with iterator	$O(n)$
set / map	—	$O(\log n)$	$O(\log n)$	$O(\log n)$
unordered_set / unordered_map	—	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg, $O(n)$ worst
stack / queue	top/front only, $O(1)$	$O(1)$	—	—
priority_queue	top only, $O(1)$	$O(\log n)$	—	—

8.2 Algorithms (from <algorithm>)

Algorithm	Complexity	Precondition
sort	$O(n \log n)$	—
stable_sort	$O(n \log n)$	—
binary_search / lower_bound / upper_bound (on iterators)	$O(\log n)$	Range must be sorted; requires random-access iterators for the $O(\log n)$ guarantee
find / count / count_if	$O(n)$	—
accumulate	$O(n)$	—
min_element / max_element	$O(n)$	—
reverse	$O(n)$	—
rotate	$O(n)$	—
unique (on a sorted range)	$O(n)$	Range should be sorted to remove all duplicates
next_permutation (single call)	$O(n)$ worst case, $O(n)$ amortized over a full cycle	—

WARNING — The set/map member-function exception, revisited

As emphasized in Part IV: the free functions `lower_bound/upper_bound` require random-access iterators to achieve $O(\log n)$ — calling them on a `set` or `map` (which have bidirectional, not random-access, iterators) silently falls back to $O(n)$ per call. Always use the member-function version (`s.lower_bound(x)`) on those containers.

Chapter 9 — Complexity Case Studies: Sorting, Searching,

Graphs, Trees, DP

9.1 Sorting algorithms (previewing Part VI)

Algorithm	Best	Average	Worst	Space
Bubble / Selection / Insertion sort	$O(n) / O(n^2) / O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Counting sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$

NOTE — Why `sort()` is safe against worst-case attacks

A naive quicksort can be forced to $O(n^2)$ by an adversarial input targeting its pivot strategy — a real historical issue on some judges with hand-written quicksorts. C++'s `std::sort` uses introsort, which detects excessive recursion depth and switches to heapsort, guaranteeing $O(n \log n)$ worst case — one more reason to always prefer the standard library's `sort()` over a hand-rolled one.

9.2 Searching (previewing Part VI)

Algorithm	Complexity	Precondition
Linear search	$O(n)$	None
Binary search	$O(\log n)$	Sorted array (or a monotonic predicate — 'binary search on the answer')

9.3 Graphs (previewing Part XIII)

Algorithm	Complexity	Notes
BFS / DFS	$O(V + E)$	V = vertices, E = edges; each visited once, each edge examined once
Dijkstra (with a binary heap)	$O(E \log V)$	Each edge relaxation may push onto the heap
Bellman-Ford	$O(V * E)$	Relaxes every edge, up to $V-1$ times
Floyd-Warshall	$O(V^3)$	All-pairs shortest paths via triple-nested loop
Kruskal's MST	$O(E \log E)$	Dominated by sorting the edges
DSU operations (with path compression + union by size)	$O(\alpha(n))$ amortized	α is the inverse Ackermann function — effectively constant

These are previews — Part XIII derives each of these from first principles. The point here is pattern recognition: $V+E$ complexity means "visit every node and edge a constant number of times," a shape that recurs constantly once you know to look for it.

9.4 Dynamic programming (previewing Part XV)

A DP's time complexity is almost always (number of distinct states) $\times$ (work per transition). This single formula demystifies most DP complexity questions immediately:

DP	States	Work/transition	Total
Fibonacci (memoized)	$O(n)$	$O(1)$	$O(n)$
0/1 Knapsack	$O(n * W)$ (items x capacity)	$O(1)$	$O(n * W)$
Longest Common Subsequence	$O(n * m)$ (two string lengths)	$O(1)$	$O(n * m)$
Matrix Chain Multiplication	$O(n^2)$ (interval $[i,j]$)	$O(n)$ (try every split point)	$O(n^3)$

KEY IDEA — The states x transition formula

Before writing any DP solution, identify the number of distinct states first (this is usually visible directly from the problem's dimensions) and the cost to compute one state's answer from others. Multiplying the two gives the complexity immediately, letting you sanity-check a DP idea against the constraints before implementing it — exactly the same discipline as step 6 of the Part II framework, applied specifically to DP.

Chapter 10 — Hidden-Complexity Traps

Each of these has appeared, in disguise, somewhere earlier in this book — collected here as a single reference of "innocent-looking code that is secretly slower than it appears."

Innocent-looking code	Hidden cost	Why
<code>find(v.begin(), v.end(), x)</code> inside a loop over <code>v</code>	$O(n)$ per call -> $O(n^2)$ total	Linear search doesn't become free just because it's one line
<code>v.erase(v.begin())</code> inside a loop	$O(n)$ per call -> $O(n^2)$ total	Every element after the erased one shifts left
<code>s += c</code> inside a loop building a large string, in some implementations	Can be $O(n)$ per append -> $O(n^2)$ total	Reallocation/copy behaviour similar to vector, though <code>libstdc++</code> often amortizes this well
Free <code>lower_bound()/upper_bound()</code> called on a <code>set/map</code>	$O(n)$ per call instead of $O(\log n)$	Free-function binary search needs random-access iterators
Recomputing a prefix sum range from scratch inside a query loop	$O(n)$ per query -> $O(n*q)$ total	Ignoring the $O(n)$ preprocessing + $O(1)$ query pattern from Chapter 5
<code>accumulate(v.begin(), v.end(), 0)</code> on large values	Silent integer overflow, not a speed issue	The initial value's type (<code>int</code>) determines the accumulator's type
A recursive solution with no memoization on an overlapping-subproblem structure	$O(2^n)$ instead of $O(n)$ or $O(n^2)$	Recomputing the same state repeatedly, as in naive Fibonacci

Chapter 11 — Complexity Practice Drills

State the tightest Big-O complexity for each snippet below before checking the answer. These are deliberately varied and increase gradually in subtlety.

Drill 1

```
for (int i=0;i<n;i++) cout << i;
```

Answer: $O(n)$ — Why: Single loop, $O(1)$ work per iteration.

Drill 2

```
for (int i=0;i<n;i++) for (int j=0;j<n;j++) cout << i*j;
```

Answer: $O(n^2)$ — Why: Two independent nested loops both bounded by n .

Drill 3

```
for (int i=0;i<n;i++) for (int j=0;j<m;j++) cout << i*j;
```

Answer: $O(n * m)$ — Why: Two DIFFERENT bounds — do not conflate n and m .

Drill 4

```
for (int i=1;i<n;i*=3) cout << i;
```

Answer: $O(\log n)$ — Why: Multiplicative step — logarithmic, base doesn't matter for Big-O.

Drill 5

```
for (int i=0;i<n;i++) for (int j=i;j<n;j++) cout << i+j;
```

Answer: $O(n^2)$ — Why: Triangular bound — still $O(n^2)$, constant factor differs (Section 3.3).

Drill 6

```
for (int i=0;i<n;i++) { int j=1; while(j<n) j*=2; }
```

Answer: $O(n \log n)$ — Why: Outer $O(n)$, inner $O(\log n)$, nested and independent of each other $\rightarrow$ multiply.

Drill 7

```
sort(v.begin(), v.end());
```

Answer: $O(n \log n)$ — Why: Standard library sort guarantee.

Drill 8

```
for (int x : v) if (binary_search(v.begin(), v.end(), x)) cnt++;
```

Answer: $O(n \log n)$ — Why: $O(n)$ outer loop, $O(\log n)$ binary search per element (v is sorted and has random-access iterators).

Drill 9

```
for (int x : v) if (mySet.find(x) != mySet.end()) cnt++; // mySet is  
std::set
```

Answer: $O(n \log n)$ — Why: `set::find` is a member function, correctly $O(\log n)$; n calls $\rightarrow O(n \log n)$.

Drill 10

```
for (int x : v) if (find(v.begin(), v.end(), x) != v.end()) cnt++;
```

Answer: $O(n^2)$ — Why: The free `find()` on a vector is $O(n)$ per call, called n times.

Drill 11

```
long long fact(int n){ if(n<=1) return 1; return n*fact(n-1); }
```

Answer: $O(n)$ time, $O(n)$ space — Why: Single recursive call per level; stack depth n .

Drill 12

```
long long f(int n){ if(n<=1) return n; return f(n-1)+f(n-2); }
```

Answer: $O(2^n)$ time, $O(n)$ space — Why: Two calls per level with only a -1 reduction each time; exponential nodes, but linear stack depth.

Drill 13

```
void mergeSort(vector<int>&a,int l,int r){ if(l>=r) return; int  
m=(l+r)/2; mergeSort(a,l,m); mergeSort(a,m+1,r); merge(a,l,m,r); }  
// merge is  $O(r-l+1)$ 
```

Answer: $O(n \log n)$ — Why: Master Theorem: $a=2$, $b=2$, $d=1$, $\log_b(a)=1=d \rightarrow$ balanced case.

Drill 14

```
for(int i=0;i<n;i++) v.insert(v.begin(), a[i]);
```

Answer: $O(n^2)$ — Why: `insert()` at the front of a vector is $O(n)$, done n times.

Drill 15

```
for(int i=0;i<n;i++) dq.push_front(a[i]); // dq is a deque
```

Answer: $O(n)$ — Why: deque `push_front` is $O(1)$, unlike a vector's.

Drill 16

```
unordered_map<int,int> m; for(int x: v) m[x]++;
```

Answer: $O(n)$ average — Why: Each hash-map operation is $O(1)$ average, n operations total.

Drill 17

```
for(int i=0;i<n;i++) for(int j=0;j<n;j++) for(int k=0;k<n;k++)  
c[i][j]+=a[i][k]*b[k][j];
```

Answer: $O(n^3)$ — Why: Classic triple-nested matrix multiplication.

Drill 18

```
int i=n; while(i>0) i/=2;
```

Answer: $O(\log n)$ — Why: Divisive step -> logarithmic.

Drill 19

```
for (int mask=0; mask < (1<n); mask++) { /* O(n) work per mask */ }
```

Answer: $O(n * 2^n)$ — Why: 2^n masks, each with $O(n)$ work (e.g. iterating over set bits).

Drill 20

```
string s; for(int i=0;i<n;i++) s = s + 'a';
```

Answer: $O(n^2)$ worst case in general, though often better in practice — Why: Each += on a string can trigger a reallocation-and-copy of the whole current string, in the worst analysis; amortized growth strategies often make this better in practice, similar to vector, but treat it as $O(n^2)$ unless verified otherwise.

Drill 21

```
for (auto it = s.begin(); it != s.end(); it++) cout << *it; // s is a  
set
```

Answer: $O(n)$ — Why: A single full traversal of any container with n elements is $O(n)$, regardless of the container's per-element access complexity.

Drill 22

```
priority_queue<int> pq; for(int x: v) pq.push(x); while(!pq.empty())  
{ pq.pop(); }
```

Answer: $O(n \log n)$ — Why: n pushes and n pops, each $O(\log n)$.

Drill 23

```
for(int i=0;i<n;i++) cout << v[i]; for(int i=0;i<n;i++) cout <<  
v[i];
```

Answer: $O(n)$ — Why: Two SEQUENTIAL loops of size n add: $O(n)+O(n)=O(n)$, not $O(n^2)$ — they are not nested.

Drill 24

```
int gcdSum=0; for(int i=1;i<=n;i++) for(int j=1;j<=n;j++) gcdSum +=  
__gcd(i,j);
```

Answer: $O(n^2 \log(\min(n,n)))$ — Why: n^2 pairs, each `__gcd` call costs $O(\log(\min(i,j)))$ via the Euclidean algorithm (Part XVII).

Drill 25

```
for (int i = 0; i < n; i++) { vector<int> temp = v;  
sort(temp.begin(), temp.end()); }
```

Answer: $O(n^2 \log n)$ — Why: A fresh $O(n)$ copy plus an $O(n \log n)$ sort, repeated n times.

Drill 26

```
int x = 0; for (int i = 0; i < 1000000; i++) x += i; // n does not  
appear at all
```

Answer: $O(1)$ — Why: A fixed number of iterations regardless of any input size n is a constant, not a function of n .

Drill 27

```
bool isPrime(int n){ for(int i=2;i*i<=n;i++) if(n%i==0) return false; return true;}
```

Answer: $O(\sqrt{n})$ — Why: Loop runs while $i*i \leq n$, i.e. up to about $\sqrt{n}$ iterations.

Drill 28

```
for(int i=0;i<n;i++){ if(i%2==0) for(int j=0;j<n;j++) cout<<j; }
```

Answer: $O(n^2)$ — Why: The inner $O(n)$ loop runs for roughly half of the outer iterations — still $O(n) * O(n/2) = O(n^2)$ after dropping the constant factor.

Drill 29

```
map<int,int> m; for (int x : v) m[x]++; for (auto &[k,val] : m) cout<<k<<val;
```

Answer: $O(n \log n)$ — Why: n map insertions at $O(\log n)$ each, plus an $O(\text{distinct keys})$ traversal, dominated by the $O(n \log n)$ insertion phase.

Drill 30

```
int f(int n){ if(n==0) return 0; return 1 + f(n/2); }
```

Answer: $O(\log n)$ time, $O(\log n)$ space — Why: Input halves each call $\rightarrow$ logarithmic depth and, since it's a single chain of calls, logarithmic total work too.

Part V — Master Revision

Must Remember

- Sequential loops add their costs; nested (dependent) loops multiply theirs.
- A loop variable multiplied/divided by a constant factor each iteration gives $O(\log n)$ iterations, not $O(n)$.
- A recursion's time complexity comes from its recursion tree (total nodes); its space complexity comes from its maximum depth (the longest single chain of calls) — these can differ dramatically, as in naive Fibonacci ($O(2^n)$ time, $O(n)$ space).
- The Master Theorem applies only to $T(n) = a \cdot T(n/b) + O(n^d)$; compare d against $\log_b(a)$ to classify the recurrence.
- Amortized analysis bounds TOTAL work across a whole sequence of operations, then divides by the count — it is a guarantee for every input sequence, unlike average-case analysis.
- A DP's complexity is (number of distinct states) $\times$ (work per transition) — compute both before implementing.
- Always design for the worst case in competitive programming; judges construct data specifically to expose fragile 'usually fine' approaches.

Master Complexity Cheat Sheet

Code shape	Complexity
Single loop over n	$O(n)$
Two independent nested loops over n	$O(n^2)$
Triangular nested loop (j starts at i)	$O(n^2)$
Loop variable $\times$ or $\div$ constant	$O(\log n)$
Outer $O(n)$ loop with inner $O(\log n)$ work	$O(n \log n)$
Recursion halving input, $O(1)$ combine (binary search)	$O(\log n)$
Recursion halving input, $O(n)$ combine (merge sort)	$O(n \log n)$
Recursion reducing input by 1, two calls (naive Fibonacci)	$O(2^n)$
Enumerating all subsets of n items	$O(2^n)$
Enumerating all permutations of n items	$O(n!)$
Triple nested loop over n (matrix multiplication)	$O(n^3)$
vector <code>push_back</code> over n elements	$O(n)$ amortized total, $O(1)$ amortized per call
DSU with path compression + union by size, m operations	$O(m * \alpha(n)) \approx O(m)$

Interview-Style Questions

- 1 Why does naive recursive Fibonacci run in $O(2^n)$ time but only $O(n)$ space?
- 2 State the Master Theorem's three cases and apply it to merge sort.
- 3 Explain, using the accounting-method intuition, why `vector::push_back` is amortized $O(1)$ despite occasional $O(n)$ reallocations.
- 4 Why should you always assume worst-case behaviour when designing a competitive programming solution?

- 5 Given a DP problem with states indexed by (i, j) where i, j range up to n , and an $O(n)$ transition, what is the overall time complexity?

NOTE — Coming up in Part VI

Part VI — Searching and Sorting puts this complexity toolkit to immediate use: implementing and analyzing every classic sort from scratch, then going deep on binary search — including the binary-search-on-the-answer pattern previewed in Part IV — building the exact skill needed for difficult LeetCode and Codeforces binary-search problems.